Software Architecture and Design Specification

Project: Web Based ML Model Evaluator

Version: 1.0

Authors: Sanyam Devan Hiran, Shreya Patil, Satvik Phadke, Shreya Ajith (Team 13)

Date: 05-10-2026

Status: Draft

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification describes the architecture and design of the **Web Based ML Model Evaluator**. It explains the system components, data flow, interfaces, and security considerations involved in uploading datasets, configuring ML tasks, training models, and evaluating results.

1.2 Scope

The system provides a web-based pipeline for **CSV upload, data preprocessing, ML task configuration, model execution, and result visualization**. It supports Classification, Regression, and Clustering using algorithms such as Logistic Regression, Decision Tree, Random Forest, KNN, and K-Means. The system also provides relevant performance metrics and visualizations.

1.3 Audience

This document is intended for **developers, QA and test engineers, security auditors, instructors, students, and maintenance teams** involved in developing, testing, evaluating, or maintaining the system.

1.4 Definitions

- **ML:** Machine Learning
- **CSV:** Comma-Separated Values
- **API:** Application Programming Interface
- **JSON:** JavaScript Object Notation

- **PCA:** Principal Component Analysis
- **STRIDE:** Security threat-modeling framework
- **TSX:** TypeScript XML used for the frontend
- **SADS:** Software Architecture and Design Specification
- **SRS:** Software Requirements Specification
- **RTM:** Requirements Traceability Matrix

2. Document Overview

2.1 How to use this document

This Software Architecture and Design Specification serves as the structural and design document for this project. It outlines the architectural components (high level design) required for the implementation, verification, and maintenance of the application, including:

- Architectural and Analysis Models: High level component design, sequence of execution, and ownership assignments.
- Design and UML Diagrams: Step by step UML sequence diagram detailing data payload and system interactions across the frontend and backend components.
- Threat Models and Security Architecture: Uses the STRIDE framework, detailing about mitigating Denial of Service (DoS), data leakage, and parameter validation.
- API Specifications and Interfaces: Comprehensive endpoint definitions, request/response formats, status codes, and centralized error handling contracts.

Documents to be viewed (role based):

- Developers: Section 3 (Architecture) and Section 4 (Design) to understand the structure of the project and build based on the requirements.
- QA & Test Engineers: Utilize Section 3.8 (Traceability) and Section 4.3 and 4.4 (API & Error Handling) to align test scenarios with system components.
- Security Auditors & Evaluators: Review Section 3.9 (Security Architecture) and Section 3.1 (Constraints) regarding session memory management and input size enforcement.

2.2 Related Documents

This specification document is related to the following project documents:

- Software Requirements Specification (SRS): Defines the complete set of functional requirements (FR1 through FR13), non functional constraints (30 second response latency, 10 MB file size limit), and business rules governing the Web Based ML Model Evaluator.
- Software Test Plan (STP): Details the testing strategy, test coverage across all four core modules, execution schedules, entry/exit criteria, and security/performance verification procedures.
- Requirements Traceability Matrix (RTM): Maps every functional requirement (FR) and non functional requirement (NFR) to its corresponding architectural module, design component, code file reference, and test case ID to ensure complete lifecycle coverage.

3. Architecture

3.1 Goals & Constraints Goals: Provide an easy-to-use pipeline executing ingestion, scaling, fitting, and metrics generation within a 30-second response latency per run. Constraints: The system strictly restricts file uploads to a maximum of 10 MB to prevent server slowdowns. No long-term external database persistence is used; all execution is handled via session-based in-memory processing.

3.2 Stakeholders & Concerns Customers (Students/Teachers): Require a guided, simple interface without the overhead of setting up local code. Developers (Team 13): Require a highly decoupled architecture divided exactly into four independent modules to allow Shreya A., Satvik P., Shreya P., and Sanyam H. to work concurrently without merge conflicts.

3.3 Component Architecture & Data Flow

Pipeline Stage	Module Name	Primary Owner	Core Technology	Input	Output / Data Contract
Stage 1	Data Ingestion & Preprocessing	Shreya Ajith	Rust, Polars	Raw CSV File (< 10 MB)	Standardized Cleaned Data JSON

Stage 2	ML Task Routing & Configuration	Satvik Phadke	TSX Frontend, Rust APIs	Cleaned Data JSON, User UI Input	Task Configuration Struct
Stage 3	Algorithm Execution & Metrics	Shreya Patil	Rust, Linfa	Cleaned Data JSON + Config Struct	Raw Results JSON Object
Stage 4	Visualization, Comparison & Errors	Sanyam Deven Hiran	TSX, Recharts/Plo		

3.4 Component Descriptions

To ensure absolute independence among the four developers, the system is architected into four decoupled pipeline modules spanning the TSX frontend and Rust backend, communicating strictly through standardized JSON data contracts:

- Data Ingestion and Preprocessing Module: Handles CSV upload, strict 10MB size validation, and file formatting. Analyzes the dataset via Rust to return row/column counts, data types, and a 5-row preview. Manages missing value cleaning using Polars. Abstraction: Receives a raw file and outputs a standardized "Cleaned Data Object".
- ML Task Routing & Configuration Module: Manages user selections in the TSX UI for the problem type (Classification, Regression, Clustering). Handles target/feature column selections, clustering parameters, and train/test split configurations. Abstraction: Consumes the "Cleaned Data Object" metadata and outputs a "Task Configuration Struct" to the Rust API.
- Algorithm Execution & Metric Computation Module: Executes the mathematical model fitting strictly in Rust (Logistic Regression, Decision Tree, Random Forest, KNN, K-Means). Computes raw numerical metrics (Accuracy, F1-Score, MAE, R2,

Silhouette Score). Abstraction: Consumes the "Cleaned Data Object" and "Task Configuration Struct" to output a "Raw Results Object".

- Visualization, Comparison & Error Handling Module: Transforms raw metrics into a TSX Model Comparison Dashboard. Renders Confusion Matrices and 2D PCA scatter plots using frontend charting libraries. Intercepts invalid execution states across all Rust backend routes and returns structured, human-readable error messages to the UI.

3.5 Chosen Architecture Pattern and Rationale

A Sequential Pipeline / Micro-Core Architecture was chosen. A monolithic design would cause severe development bottlenecks for a 4-person team. By enforcing strict data contracts between the four distinct layers, each team member can develop, unit test, and mock their TSX UI components and Rust backend handlers concurrently.

3.6 Technology Stack & Data Stores Backend: Rust, Axum or Actix-Web (API framework), Polars (for high-performance data manipulation replacing pandas), Linfa or SmartCore (for ML algorithms replacing scikit-learn). Storage: Transient session-based memory only. Host memory buffers are purged of uploaded CSVs and model artifacts upon session termination. Frontend: TSX (React/TypeScript/Next.js) supporting modern browsers (Chrome, Firefox, and Edge). Plotly.js or Recharts for rendering PCA scatter plots and matrices.

3.7 Risks & Mitigations Risk: Processing delays exceeding the 30-second pipeline response latency. Mitigation: Utilizing Rust's zero-cost abstractions and Polars' multi-threaded execution guarantees processing well under the time limit. Perform early performance testing with datasets nearing the 10 MB limit. Risk: Unstructured or corrupted CSV files breaking the pipeline. Mitigation: The Rust Error Handling middleware will safely catch panics, intercept invalid states, and return structured error messages to the TSX frontend.

3.8 Traceability to Requirements R1 (FR-1, FR-2, FR-3) -> Data Ingestion & Preprocessing Module R2 (FR-4, FR-5, FR-6, FR-7) -> ML Task Routing & Configuration Module R3 (FR-8, FR-9, FR-10) -> Algorithm Execution Module R4 (FR-11, FR-12, FR-2 Error Handling) -> Visualization, Comparison & Error Handling Module

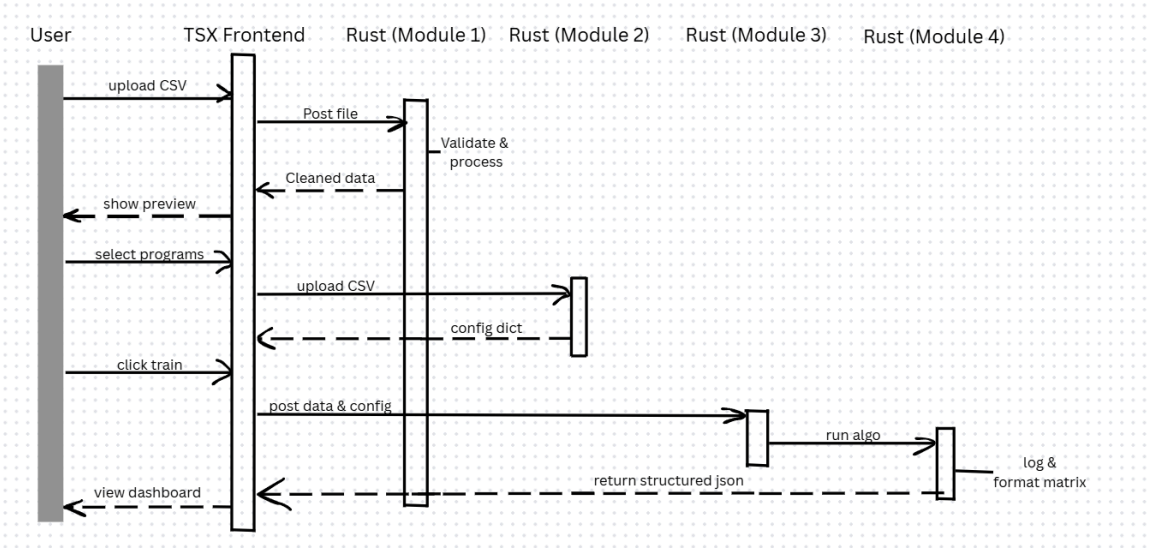
3.9 Security Architecture Threat Modeling (STRIDE): DoS -> rate limiting and enforcing strict file size boundaries with a hard 10 MB restriction. Data Leakage / Info disclosure -> purging host memory of uploaded CSVs upon session termination. Elevation of Privilege / Injection -> Validating input parameters strictly via Rust's strong type system and ensuring error messages do not expose sensitive host server details.

4. Design

4.1 Design Overview

The TSX frontend application and Rust backend are designed as a step-by-step wizard. The strict four-module pipeline prevents the execution module from firing before ingestion and configuration are fully validated, ensuring predictable data flows.

4.2 UML Sequence Diagram (Execution Flow)



4.3 API Design

Endpoint: /api/upload

Method: POST

Request: multipart/form-data {file: .csv}

Response: {status: string, row_count: u32, col_names: Vec, missing_vals: HashMap, preview: Vec}

Errors: 400 Invalid File Type, 413 Payload Too Large (>10MB).

Endpoint: /api/execute_model

Method: POST

Request: {problem_type: string, target_col: string, features: Vec, algo: string, test_split: f32}

Response: {metrics: HashMap, plot_data: Vec}

Errors: 422 Unprocessable Entity.

4.4 Error Handling, Logging & Monitoring A centralized Rust middleware interceptor catches invalid file formats, unsupported algorithm parameters, and missing data. It maps Rust Result/Option types into human-readable JSON error messages without exposing host server details to the TSX client.

4.5 UX Design The TSX UI follows a guided, clear process: upload -> select columns -> train model -> display results. The frontend is strictly typed with TypeScript and runs smoothly on Google Chrome, Firefox, and Microsoft Edge without requiring extra browser plugins.

4.6 Open Issues & Next Steps Completion of unit testing across the 4 separated pipelines, including Cargo tests for Rust and Jest/React Testing Library for TSX. Integration testing for the API interaction between the frontend and the Rust backend. Filling out the Requirements Traceability Matrix (RTM) completely during testing.

5. Appendices

5.1 Glossary

Term	Meaning
ATM	Automated Teller Machine
PIN	Personal Identification Number
PCI-DSS	Payment Card Industry Data Security Standard
TLS	Transport Layer Security
ADR	Architecture Decision Record

5.2 References

Reference	Full Title
IEEE 42010	ISO/IEC/IEEE 42010: Systems and software engineering, Architecture description
OWASP	Open Worldwide Application Security Project (Top Ten and security guidelines)
NIST SP 800-160	Systems Security Engineering: Considerations for a Multidisciplinary Approach

5.3 Tools

Tool	Purpose
PlantUML	Text-based UML diagram generation (e.g. sequence diagram in Section 4.2)
draw.io	Architecture and pipeline diagrams
Swagger	API specification and documentation (Section 4.3)